Software Requirements Specification: Doctor Portfolio and Appointment Request Website

Version 1.0 | October 21, 2026 | Prepared by Fazly Rahi

1. Introduction

1.1 Purpose

This document specifies the requirements for the Doctor Portfolio and Appointment Request Website, version 1.0. It describes what the system does, who uses it, how it is built and hosted, and the limits it operates under. It is written for the developer, the site owner (the doctor) and any future maintainer or reviewer, and follows the structure recommended by IEEE 830 and ISO/IEC/IEEE 29148.

1.2 Scope

The product is a single-page, responsive website that presents a doctor's professional profile and lets visitors request a consultation time online. It is a demonstration site with sample content for the doctor **Dr. Saifun Mahdi Antara**. All profile details must be verified before real use.

In scope

- Profile presentation: hero, about, specialities, experience, certificates and contact sections
- An appointment request form with validation
- Storing requests in Firebase Firestore
- Email notification to the doctor through EmailJS
- A limit of 6 appointment requests per day
- Hosting on Netlify with automatic deployment from GitHub

Out of scope

- User accounts or login
- Online payment
- Confirmed bookings or calendar management
- An admin dashboard
- Automatic confirmation emails to patients
- Storing medical records
- Video consultation

1.3 Definitions, Acronyms and Abbreviations

Term	Meaning
SRS	Software Requirements Specification
Firestore	Cloud Firestore, the NoSQL database provided by Firebase

Term	Meaning
EmailJS	A service that sends email directly from browser JavaScript without a custom server
Netlify	A platform that hosts static websites and deploys them automatically
CI/CD	Continuous integration and deployment: code pushed to GitHub is published automatically
HTTPS	Encrypted version of HTTP used for all traffic between the browser and the services
Appointment request	A visitor's preferred date and time. It is not a confirmed appointment.
Honeypot	A hidden form field that humans never fill but spam bots often do
Transaction	A database operation where several reads and writes succeed together or not at all
Responsive	A layout that adapts to desktop, tablet and phone screens

1.4 References

- IEEE 830-1998, Recommended Practice for Software Requirements Specifications
- ISO/IEC/IEEE 29148:2018, Requirements Engineering
- Firebase documentation: Cloud Firestore and Security Rules (Web SDK v10)
- EmailJS documentation: Browser SDK v4
- Netlify documentation: Continuous deployment
- Web Content Accessibility Guidelines (WCAG) 2.1

1.5 Document Overview

Section 2 gives the overall picture of the product. Section 3 lists the functional requirements. Section 4 covers external interfaces, Section 5 the data, and Section 6 the non-functional requirements. Section 7 shows how the system behaves, Section 8 explains deployment and maintenance, and Section 9 lists limitations and future work.

2. Overall Description

2.1 Product Perspective

The system is a standalone static website with no custom server. The visitor's browser loads the pages from Netlify and then talks directly to two third-party services: Firebase Firestore for storage and EmailJS for email.

Layer	Technology	Role

Layer	Technology	Role
Frontend	HTML5, CSS3, JavaScript (ES6, no framework)	Page structure, styling and form logic
Database	Firebase Firestore (Web SDK 10.8.0 loaded from CDN)	Stores appointment requests and the daily counter
Email	EmailJS (Browser SDK v4) with a Gmail service	Sends a notification email to the doctor
Hosting	Netlify	Serves the site over HTTPS
Version control	GitHub, connected to Netlify	Stores source code. Each commit is deployed automatically.

Data flow: Browser → Netlify (page files). Browser → Firestore (save request). Browser → EmailJS → Gmail → doctor's inbox (notification).

2.2 Product Functions

- Present the doctor's profile, specialities, experience and credentials
- Let visitors request a consultation date and time
- Validate the request before sending it
- Save each request securely in the database
- Limit requests to 6 per day
- Notify the doctor by email
- Show clear success and error messages to the visitor
- Work on desktop, tablet and mobile devices

2.3 User Classes and Characteristics

User class	Description	Technical skill
Visitor / Patient	Any member of the public who views the profile and submits a request	Basic
Doctor (site owner)	Receives notification emails and reviews saved requests in the Firebase console	Basic
Developer / Maintainer	Edits the code, manages Firebase, EmailJS, Netlify and GitHub settings	Intermediate

2.4 Operating Environment

- Current versions of Chrome, Edge, Firefox and Safari on desktop and mobile
- JavaScript enabled and support for ES modules
- An active internet connection
- Screen widths from about 320 px (small phone) to large desktop monitors

2.5 Design and Implementation Constraints

- The project uses free plans. The EmailJS dashboard shows a quota of 200 requests, and Firestore has its own free usage limits.
- There is no server-side code. Web keys for Firebase and EmailJS are visible in the browser by design, so protection depends on Firestore security rules and EmailJS settings.
- Email is sent through a Gmail account connected to EmailJS.
- The site is static, so all logic runs in the visitor's browser.

2.6 Assumptions and Dependencies

- The doctor checks the notification inbox and the Firebase console regularly.
- Firebase, EmailJS, Netlify and GitHub stay available and keep offering free plans.
- Visitors give truthful contact details.
- All profile content is sample content until the doctor verifies and replaces it.

3. System Features and Functional Requirements

Priority: **Must** means required for release. **Should** means expected but the system still works without it.

3.1 Navigation

ID	Requirement	Priority
FR-1	The header shall show the doctor's name and links to About, Specialities, Experience, Certificates and Contact, plus a Book Consultation button.	Must
FR-2	The header shall stay at the top of the screen while the visitor scrolls.	Should
FR-3	Clicking a link shall scroll smoothly to its section.	Should
FR-4	On screens 760 px wide or less, the links shall collapse into a menu button that opens and closes the list, and the list shall close after a link is chosen.	Must
FR-5	The menu button shall update its expanded state and label for screen readers.	Should

3.2 Profile Content

ID	Requirement	Priority
FR-6	The hero section shall show a headline, short introduction, the buttons Book a Consultation and Explore My Profile, key details (qualification, speciality, location) and a portrait.	Must
FR-7	The About section shall present the doctor's biography and qualification summary.	Must
FR-8	The Specialities section shall show three cards: General Medicine, Preventive Care and Health Consultation.	Must

ID	Requirement	Priority
FR-9	The Experience and Education section shall show a timeline of practice, training and degree.	Must
FR-10	The Certificates section shall show three document cards, each with a Request information link to the contact section.	Must
FR-11	The Contact section shall show the doctor's name, location, clinic hours and an Email the Doctor button.	Must
FR-12	The footer shall show the current year (set automatically), a back-to-top link and a demonstration disclaimer.	Must
FR-13	Sample content shall be marked with notes stating it must be verified before publication.	Should

3.3 Appointment Request Form

ID	Requirement	Priority
FR-14	The form shall collect: full name, email address, phone number, preferred date, preferred time, an optional message and a privacy consent checkbox.	Must
FR-15	Name, email, phone, date, time and consent shall be required. Limits: name 100, email 254, phone 25 and message 500 characters.	Must
FR-16	The date picker shall not allow dates before today, using the visitor's local date.	Must
FR-17	The time shall be chosen from fixed slots: 09:00, 10:00, 11:00, 14:00, 15:00 and 16:00.	Must
FR-18	The visitor shall tick the consent checkbox before sending.	Must
FR-19	The browser shall validate all fields before submission, and invalid fields shall be highlighted with a message.	Must
FR-20	A hidden honeypot field shall exist. If it contains text, the submission shall be silently ignored.	Should
FR-21	While sending, the submit button shall be disabled and show Sending Request. It shall always return to normal afterwards, even after an error.	Must
FR-22	The form shall show one of the status messages listed in the table below.	Must
FR-23	After the request is saved, the form shall be cleared.	Must
FR-24	The form shall display a privacy note (do not include medical records)	Must

ID	Requirement	Priority
	and a disclaimer that submitting does not confirm an appointment.	

Status messages (FR-22)

Situation	Message shown	Colour
Saved and email sent	Your request was saved and the email notification was sent.	Green
Saved but email failed	Your request was saved, but the email notification failed. Please contact the clinic if needed.	Red
Could not save	We couldn't save your request. Please try again.	Red
Daily limit reached	Today's appointment requests are full. Please try again tomorrow or contact the clinic directly.	Red

3.4 Daily Request Limit

ID	Requirement	Priority
FR-25	The system shall accept at most 6 appointment requests per calendar day, using Bangladesh time (Asia/Dhaka).	Must
FR-26	The system shall check the daily count and save the request in a single transaction, so two simultaneous visitors cannot exceed the limit.	Must
FR-27	When the limit is reached, the system shall save nothing, send no email and show the daily limit message.	Must
FR-28	The counter shall restart each day by using a new document named after the date.	Must
FR-29	The limit shall be enforced in the browser code and again by the Firestore security rules.	Must

3.5 Data Storage

ID	Requirement	Priority
FR-30	Each accepted request shall be saved as a document in the appointments collection with a server timestamp.	Must
FR-31	The save shall happen before the email is sent. If the save fails, no email shall be sent.	Must
FR-32	Visitors shall not be able to read, change or delete saved requests from the website.	Must

3.6 Email Notification

ID	Requirement	Priority
FR-33	After a successful save, the system shall send an email through EmailJS to the doctor containing the patient's name, email, phone, preferred date, preferred time and message.	Must
FR-34	The email subject shall read New Appointment Request from followed by the patient's name.	Should
FR-35	If the email fails, the saved request shall be kept and the visitor shall be told the request was saved but the email failed.	Must

4. External Interface Requirements

4.1 User Interfaces

- Clean medical design with a teal primary colour on white and soft grey-blue backgrounds, using the Segoe UI font family with Arial as fallback
- Responsive layout with breakpoints at 1000 px (tablet), 760 px (mobile navigation) and 420 px (small phones)
- Buttons at least 48 px tall for easy touch use
- Visible keyboard focus outlines on all interactive elements
- Reduced-motion support: animations and smooth scrolling are turned off for visitors who request it

4.2 Hardware Interfaces

None beyond the visitor's own device (computer, tablet or phone).

4.3 Software Interfaces

Interface	Purpose	Details
Cloud Firestore	Store requests and the daily counter	Web SDK 10.8.0 (modular, loaded from gstatic CDN). Collections: appointments and dailyCounts.
EmailJS	Send the doctor's notification	Browser SDK v4 (loaded from jsDelivr CDN). One Gmail service and one template named Contact Us.
Netlify	Host and serve the website	Static files published over HTTPS.
GitHub	Source control and deployment trigger	Repository connected to Netlify.

4.4 Communication Interfaces

All communication uses HTTPS. The browser contacts firestore.googleapis.com for database operations and api.emailjs.com to send email.

5. Data Requirements

5.1 Collection: appointments

Field	Type	Rule
patientName	String	Required, up to 100 characters
patientEmail	String	Required, up to 254 characters
patientPhone	String	Required, up to 25 characters
appointmentDate	String	Required, format YYYY-MM-DD, not in the past
appointmentTime	String	Required, one of the six fixed slots
patientMessage	String	Optional, up to 500 characters, may be empty
dayKey	String	Date the request was submitted, YYYY-MM-DD in Bangladesh time
createdAt	Timestamp	Set by the server when saved

5.2 Collection: dailyCounts

Field	Type	Rule
Document ID	String	The date, YYYY-MM-DD
count	Number	Starts at 1 and rises by exactly 1 per request, never above 6

5.3 Security Rules Summary

- Creating an appointment is allowed only when it contains the expected fields within the size limits and the daily counter rises by exactly 1 in the same write.
- Reading, updating and deleting appointments is not allowed for website visitors. The doctor views them in the Firebase console.
- The daily counter may be read, created with count 1, or increased by 1 up to 6. It cannot be deleted from the website.

5.4 Personal Data and Privacy

ID	Requirement	Priority
DR-1	The system shall collect only name, email, phone, preferred date and time, and an optional message.	Must

ID	Requirement	Priority
DR-2	The form shall tell visitors not to enter medical records or sensitive health details.	Must
DR-3	The visitor shall give consent before the details are used to respond.	Must
DR-4	The site owner should define how long requests are kept and delete older ones.	Should

6. Non-Functional Requirements

ID	Category	Requirement
NFR-1	Performance	The site is static and framework-free. Target: main content visible within 3 seconds on a normal broadband connection, and form feedback within 5 seconds under normal conditions.
NFR-2	Security	All traffic uses HTTPS. Firestore rules validate every write. No private key or password appears in browser code. The honeypot reduces simple bot spam.
NFR-3	Privacy	Personal data is collected only with consent and used only to respond to the request.
NFR-4	Usability	Labels are clear, required fields are marked with an asterisk, and messages tell the visitor what happened and what to do next.
NFR-5	Accessibility	Semantic HTML, labelled inputs, a live status region for form messages, visible focus outlines and reduced-motion support.
NFR-6	Compatibility	Works on current Chrome, Edge, Firefox and Safari, on desktop and mobile.
NFR-7	Reliability	A request is saved before any email is attempted. An email failure never loses a saved request, and the submit button always recovers.
NFR-8	Maintainability	HTML, CSS and JavaScript are in separate files with commented sections. All changes are tracked in GitHub.
NFR-9	Capacity	A maximum of 6 requests per day is about 180 emails per month, which fits within the 200-request EmailJS quota.
NFR-10	Availability	Depends on Netlify, Firebase and EmailJS. If one is down, the site shows the matching error message.

7. System Models

7.1 Appointment Request Flow

1. The visitor fills in the form and clicks Send Appointment Request.
2. The browser checks required fields and formats. If invalid, the form shows what to fix and stops.
3. If the hidden honeypot field has text, the submission is silently ignored.
4. The button is disabled and shows Sending Request.
5. The code opens a database transaction and reads today's counter.
6. If the count is already 6, the visitor sees the daily limit message and the process stops.
7. Otherwise the counter is increased and the appointment is saved together.
8. If saving fails, the visitor sees the could-not-save message and the process stops.
9. The code sends the notification email through EmailJS.
10. The visitor sees the success message, or the saved-but-email-failed message if the email step failed.
11. The form is cleared and the button returns to normal.

7.2 Use Cases

ID	Use case	Actor	Summary
UC-1	View profile	Visitor	Browses the sections and contact details
UC-2	Request appointment	Visitor	Submits preferred date and time through the form
UC-3	Receive notification	Doctor	Gets an email with the request details
UC-4	Review requests	Doctor	Opens the Firebase console and reads saved requests
UC-5	Update the website	Developer	Commits changes to GitHub and Netlify publishes them automatically

8. Deployment and Maintenance

8.1 Deployment Process

1. The developer edits the files and commits them to the GitHub repository.
2. Netlify detects the commit and publishes the new version automatically, usually within minutes.
3. The developer opens the live site and hard-refreshes (Ctrl+Shift+R) to see the update.

8.2 Configuration Checklist

Service	Required setup
Firebase	Firestore database created and the security rules published
EmailJS	Gmail service connected with permission to send email. Template uses the variables patientName, patientEmail, patientPhone, appointmentDate,

Service	Required setup
	appointmentTime and patientMessage. API access from non-browser applications is turned on and the private-key (strict) mode is turned off.
Netlify	Site published and set to public, connected to the GitHub repository
Browser code	Firebase configuration values, the EmailJS public key, service ID and template ID entered correctly

8.3 Monitoring

- EmailJS dashboard: monthly request usage and Email History
- Firebase console: stored requests, daily counters and usage
- Netlify: deploy log and site status

8.4 Acceptance Tests

Scenario	Expected result
Valid request submitted	Success message, request in Firestore, email received, form cleared
A required field left empty	Browser highlights the field and nothing is sent
A past date chosen	The date picker does not allow it
Seventh request in one day	Daily limit message, nothing saved, no email
Email service unavailable	Saved-but-email-failed message, request still stored
Database unavailable	Could-not-save message, no email, button recovers
Phone screen under 760 px	Menu button appears and opens and closes the link list
Honeypot field filled	Submission ignored without any message

9. Limitations and Future Enhancements

9.1 Known Limitations

- Free plans limit email volume (EmailJS) and database usage (Firebase).
- The daily limit is checked in the browser and by security rules, but a skilled user who edits the page code could still try to get around it. There is no CAPTCHA.
- There is no admin dashboard. Requests are read in the Firebase console.
- Patients do not receive an automatic confirmation email, and a request is not a confirmed appointment.
- Profile content, certificates and the portrait are sample or placeholder material until verified.

9.2 Future Enhancements

- Firebase App Check with reCAPTCHA for stronger spam protection
- A secure admin page with login to view, confirm or decline requests
- An automatic reply email to the patient
- Availability by date, so booked time slots disappear
- Server-side limits using Cloud Functions
- A Bangla language option
- Real verified content, photos and credentials

Appendix A: Project File Structure

```
project-root/
├── index.html
├── css/
│   └── style.css
├── js/
│   └── main.js
└── assets/
    └── doctor-placeholder image
```

Appendix B: Revision History

Version	Date	Author	Description
1.0	October 21, 2026	Fazly Rahi	First release of the SRS